

Index Compression

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Compression

In our discussion so far, we have been talking about vocabulary and postings lists but all of them were **not compressed**.

Tradeoffs of compression:

- less disk **space**
- faster transfer data from disk to **memory**
- being the data compressed, IR systems can benefit from **caching**
- decompression introduces a **penalty** during queries
 - but there are many efficient compression methods so the **penalty is small**

Reducing the index size using statistics

Many researches have demonstrated that some **preprocessing activities** can reduce the number of terms and postings.

- Stemming and case-folding reduce the number of terms by 17%
- Stemming and case-folding reduce the number of postings by 3% to 4%
- Rule of 30: 30 most common words account for 30% of the tokens in written text
 - Considering the first 150 most common words as **Stop Words** cuts 25% to 30% of Postings

Lossy and Lossless Compression

Stemming, Case Folding and Stop Word are forms of **lossy compression**.

There are many other lossy techniques that, under some conditions, can be used for compression without any reduction in effectiveness (**lossless compression**).

Estimating the number of terms

Before describing compression methods, it is important to have an estimate of the number of terms.

Oxford English Dictionary (OED) defines other than 600.000 words

But the vocabulary of most **collections is much larger than OED** (OED does not include most names of people, locations, products, or scientific entities like genes. All of these must be included in an inverted index).

We can estimate the number of terms by using the **Heaps Law**

Estimating the number of terms: the Heaps Law

Let T be the number of tokens in the collection

Let k be a parameter ranging from 30 to 100

Let b be another parameter about having value about 0.50

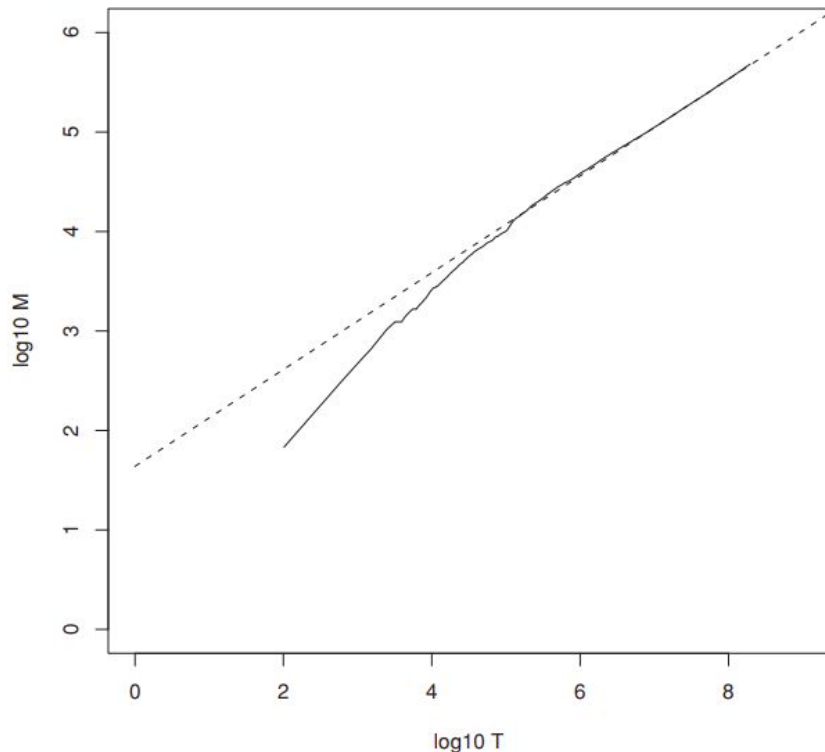
We can compute the size of the vocabulary M using the Heaps Law:

$$M = kT^b$$

Estimating the number of terms: the Heaps Law

The Heaps Law comes from the observation that the relationship between Token size T and Vocabulary size M is **linear in log-log space**.

The picture shows the relationship between M and T using a real document collection, using $k = 44$ and $b = 0.49$



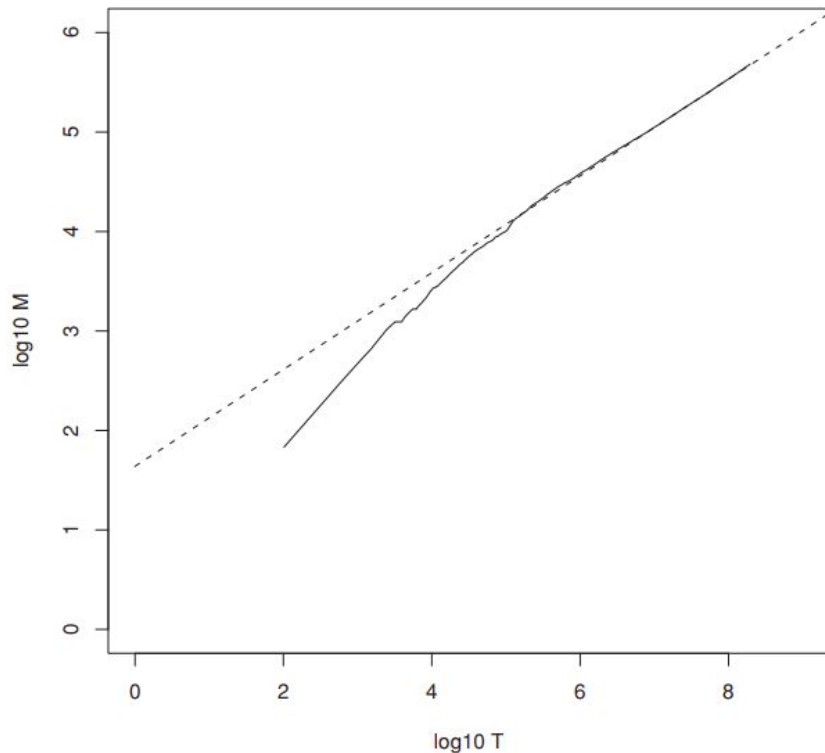
Estimating the number of terms: the Heaps Law

After processing about one million tokens, the Heaps Law predicted 38,323 of terms, whilst the real number of terms was **38,365**, which is very close to the predicted one.

$$M = kT^b$$

$$M = 44 * T^{0.49}$$

$$M = 44 * 10^{6^{0.49}} \approx 38,323$$



Estimating the number of terms: the Heaps Law

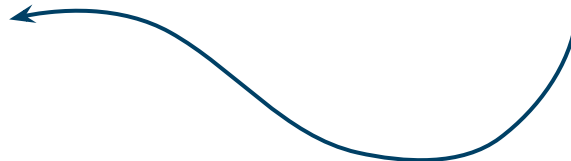
Collection for Reuters-RCV1*	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

$$T = 96,969,056$$

$$k = 50$$

$$b = 0.4875$$

$$M = kT^b = 391,249$$



* <https://archive.ics.uci.edu/dataset/259/reuters+rcv1+rcv2+multilingual+multiview+text+categorization+test+collection>

Estimating the number of terms: the Heaps Law

In conclusion, Heaps Law suggests:

1. the dictionary size continues to increase with more documents in the collection (**no maximum size is reached**)
2. the size of the dictionary is quite large for large collections

This highlights **the importance of compressed dictionary** for an effective information retrieval system

Estimate the distribution of terms

Another interesting information we need to know to build an effective algorithm for compressing postings is the **distribution of terms**.

For this purpose there is another law: the **Zipf's law**, representing a commonly used model of the distribution of terms in a collection

Estimate the distribution of terms: Zipf's law

Zipf's law states that if t_1 is the most common term, t_2 the next most common term and so on, then the collection frequency of the i -th most common term (cf_i) is

$$cf_i \propto \frac{1}{i}$$

Estimate the distribution of terms: Zipf's law

Said in other words:

The first most common word appears twice as often as the second most common word, three times as often as the third and so on.

The intuition behind this law is that **frequency decreases very rapidly with rank**.

Estimate the distribution of terms: Zipf's law

For example, consider this text:

The purpose of this paper is to investigate the applicability of Zipf's law to English text. Zipf's law states that the frequency of a word in a text is inversely proportional to its rank. In other words, the most frequent word will appear twice as often as the second most frequent word, three times as the third most frequent word, and so on.

the: 6 occurrences

of: 3 occurrences

is: 2 occurrences

Index size and estimation

As discussed, Indexes are **crucial** for a Search Engine

The larger the index, the **more complete** is the search for a user

Of course, it is not possible to know the fraction of the web indexed by a Search Engine, because there are many **dynamic web pages** making the content changed very time.

Anyway, some techniques have been defined **to estimate the ratio** of the index sizes of two Search Engines E_1 and E_2

Index size and estimation

The basic hypothesis underlying these techniques is that each Search Engine indexes a **finite subset** of the Web chosen **independently and uniformly at random**.

With this assumption, we can invoke the classical estimation technique known as the **capture-recapture method**.

This method is not new in computer science, as it is usually used for estimating population.

Capture-Recapture Method

1. Let E_1 and E_2 two Search Engines
2. Pick a page from E_1 's index (**picking phase**) and check if the link is also in E_2 's index (**checking phase**)
3. Symmetrically, pick a page from E_2 's index and test if the link is also in E_1 's index
4. Repeat steps 2 and 3 many times
5. Let x the fraction of pages in E_1 that are also in E_2 , and y the fraction of pages in E_2 that are also in E_1

Capture-Recapture Method

With these assumptions, the Capture-Recapture Method says that

$$x |E_1| \approx y |E_2|$$

$$\frac{|E_1|}{|E_2|} \approx \frac{y}{x}$$

where $|E_1|$ and $|E_2|$ are the size of the index respectively in E_1 and E_2

Page Similarity

Page Duplication

Many contents in the web are duplicated. According to some research, **30-40% of contents are duplicated.**

Many of these duplications are **legitimate copies** (pages are mirrored to provide **redundancy** and **availability**)

Some of these duplications are not legitimate copies (**plagiarism**)

Page Duplication

Having to manage duplicate pages is not convenient for Search Engines:

- More pages means larger index
- More duplicate pages means weak results (results will show different pages having the same content)

Where similarity detection is used

- Search Engine should avoid indexing and showing multiple copies of the same content (for performance and storage reasons)
- Shop Online (i.e. Amazon) can consider two customers are similar if their sets of purchased items is similar
- Moving Ratings (i.e. Netflix): two movies are considered similar if they are rated highly by many of the same customers

Detect page duplication

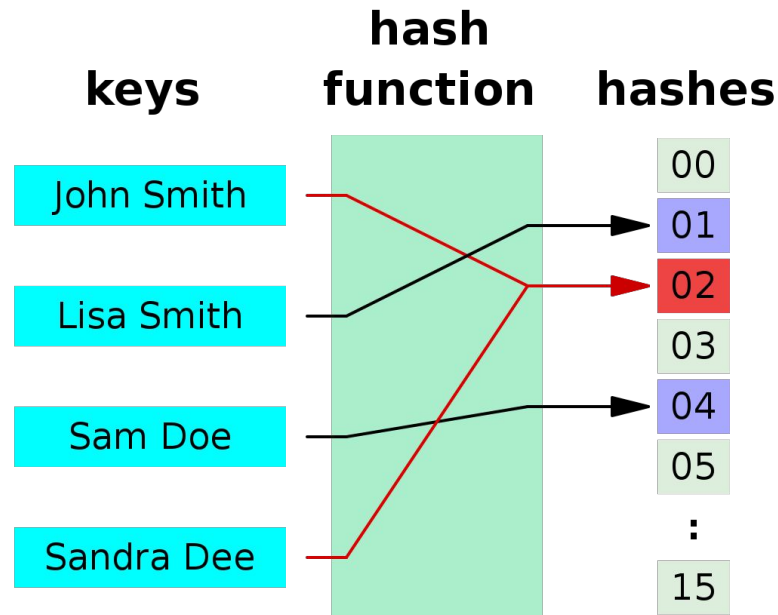
An easy way of identifying duplicated pages is to use a **fingerprint**: little variation on the page brings large variation in the fingerprint

If two pages have the same fingerprint, they are likely to be identical (**near duplication**)

How Fingerprinting works

Fingerprinting is based on Hash Functions.

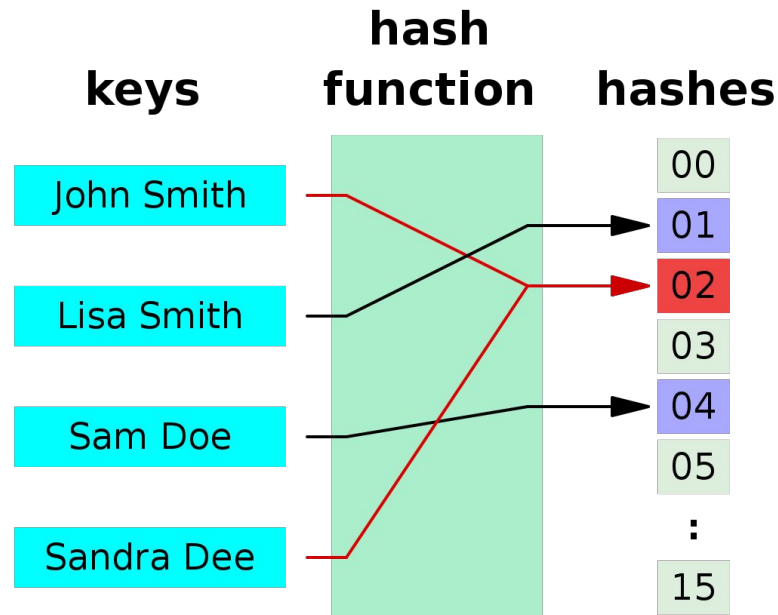
Hash Functions are able to map data of arbitrary size (named key) to a fixed-size value (named hash value or digest).



How Fingerprinting works

Being the hash valued limited in size, there might be **collisions** (two different keys produce the same hash value).

A good Hash Function distributes the possible hash-key values **approximately evenly** among buckets, to reduce the number of collision



How Fingerprinting works

Typical Hash Functions are:

- SHA (Secure Hash Algorithm)
- CRC (Cyclic Redundancy Check)
- MD5 (Message Digest 5)
- MurmurHash
- BLAKE2
- Argon2

```
$ md5sum file.pdf
eb9d2993f2c27224ffc4e44d95dd6918  file.pdf
```

Detect page duplication

This simplistic approach does not work fine when even just the publishing date and time is updated.

The solution is a techniques known as **shingling**

Detect page duplication

Shingling works on **character-level similarity**

Not “similar meaning”

Shingling

- Let k be a positive integer and d a web document
- Let define k -shingle of d be the set of all consecutive sequences of k characters in d

An example

d = "the lazy dog jumped over the quick brown fox"

k = 4

k -shingle: "the ", "he l", "e la", "laz ", "lazy", "azy ", "zy d", ...

Shingling

When two documents can be considered as similar?

Two documents are near duplicate if the sets of shingles generated from them are nearly the same

Before giving a more formal description of the Shingling, let's introduce the **Jaccard Similarity Coefficient**

Jaccard Similarity Coefficient

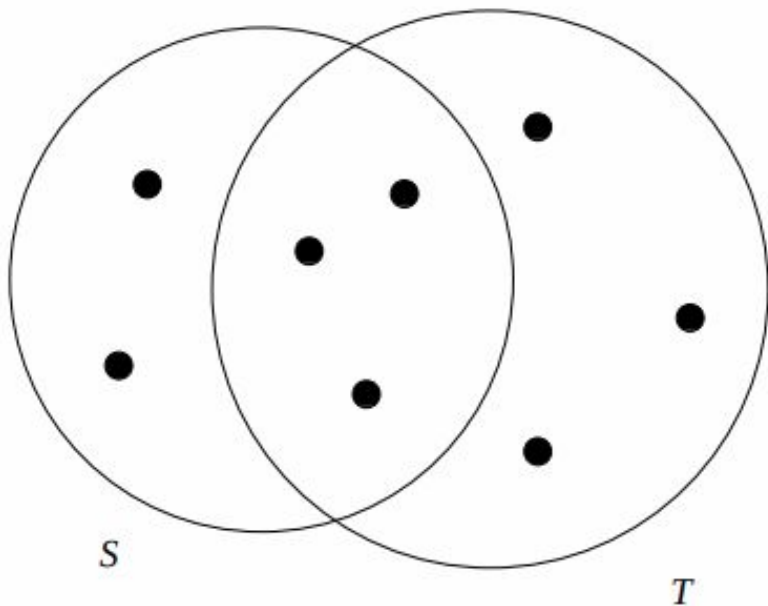
It is a statistical index for comparing similarity between two sets.

Let A and B be two sets, the Jaccard Similarity Coefficient can be computed as

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Jaccard Similarity Coefficient is equal to 1 if A and B are the same, 0 if they are different

Jaccard Similarity Coefficient



- Two sets: S and T
- $|S| = 5$
- $|T| = 6$
- $|S \cup T| = 8$
- $|S \cap T| = 3$
- $J(S, T) = 3/8$

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Shingling

- Let d_1 and d_2 be two documents
- Let $S(d_1)$ and $S(d_2)$ be the *k-shingle* sets for both documents d_1 and d_2
- The Jaccard Coefficient of both sets can be computed as

$$J(S(d_1), S(d_2)) = \frac{|S(d_1) \cap S(d_2)|}{|S(d_1) \cup S(d_2)|}$$

- If $J \geq \text{threshold}$ (for example 0.9) we can state both sets are near duplicates and the one of them can be deleted from the index

Shingling

How large k should be?

It depends on how long typical documents are. The important thing to remember is: **the larger the k, the lower the probability of finding the shingle in any document**

Which is the best value of k?

Usually $k = 5$ is fine for emails, 9 for scientific documents. Indeed, considering 26 characters in english alphabet + 1 blank space, using 5 chars we can have $27^5 = 14,348,907$ possible shingles, and a typical email is much smaller than 14 million characters long

Hashing Shingle

Instead of using Shingles as a set of strings having length k , we can use an hash function for mapping each Shingle to an integer.

So, instead of working with shingles of substrings (for example 5 or 9 bytes), we can work with shingles of integers (only 4 byte), **reducing the amount of space** to store shingles.

If we represent an integer with 32 bits, each shingle is mapped into a bucket number ranging between 0 and $2^{32}-1$

Signature

Even if we hash them to four bytes each, the space needed to store a set of shingles is very large.

If we have millions of documents, it might not be possible **to store all shingles** in main memory.

The idea is that to use a **smaller representation** of the large sets. Such a representation is named **signature**

Signature

So, for evaluating the similarity between two documents, instead of using the sets of shingles we can use the signatures of both sets

Of course, we need a way to compare the signatures of two sets and estimate the Jaccard similarity of the underlying sets from the signatures alone.

It is not possible that the signatures give the exact similarity of the sets they represent, because the size of the signature is smaller, but **the larger the signatures the more accurate the estimates**

Signature

Suppose to have 4 different documents D_1, D_2, D_3, D_4 , and we built 4 different hashing shingle sets S_1, S_2, S_3, S_4 , defined as follows $S_1 = \{a, b\}$, $S_2 = \{c\}$, $S_3 = \{b, d, e\}$, $S_4 = \{a, c, d\}$, where $U = \{a, b, c, d, e\}$ is the universal set containing all hashing shingles.

We can build the **characteristic matrix**, putting all the sets in the columns and all the elements in the rows.

Element	S1	S2	S3	S4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Characteristic matrix

Shingle sets from the documents

Shingle items

Signature

There is a 1 in row r and column c if the element for row r is a member of the set for column c .

Otherwise the value in position (r, c) is 0.

Starting from the characteristic matrix we can build the signature, by applying several hundred times an operation which is named **minhash**

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0

Signature

Minhashing:

Step 1: we make a permutation of the rows in the characteristic matrix

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

<i>a</i>			
----------	--	--	--

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

a	c		
----------	----------	--	--

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

a	c	b	
----------	----------	----------	--

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

<i>a</i>	<i>c</i>	<i>b</i>	<i>a</i>
----------	----------	----------	----------

Minhashing and Jaccard Similarity

Theorem

The probability that the minhash function for a random permutation of rows produces the same value for two sets (S_i and S_j) is equal to the Jaccard similarity of those sets.

$$\Pr(h(S_i) = h(S_j)) = J(S_i, S_j)$$

Minhashing and Jaccard Similarity

Demonstration

Let's consider the permutation of the characteristic matrix and restrict ourselves to the columns S_1 and S_4 .

All rows can be divided into three sets:

- X: rows having 1 in both columns
- Y: rows having 1 in one column and 0 in the other
- Z: rows having 0 in both columns

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

Minhashing and Jaccard Similarity

Demonstration

We can also compute those values

- x : number of rows of type X
- y : number of rows of type Y
- z : number of rows of type Z

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

Minhashing and Jaccard Similarity

Demonstration

If 1 is in both columns, it means that the corresponding element belongs to both sets (for example, the element d belongs to both S_1 and S_4 , so it is in the intersection of both sets). This means that:

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

$$J(S_1, S_4) = |S_1 \cap S_4| / |S_1 \cup S_4| = x / (x + y) = 2/3$$

Minhashing and Jaccard Similarity

Demonstration

Now, let's try to compute the probability that $h(S_1) = h(S_4)$:

Two columns have the same minhashing value if the first row, non belonging to Z, belongs to X. And being x the number of X rows and $x+y$ the total number, we can say that:

$$\Pr(h(S_1) = h(S_4)) = x / (x + y) = 2/3$$

which is equal to $J(S_1, S_4)$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

Signature Matrix

Previously, we saw how to build a Signature from a random permutation of the Characteristic Matrix. But if we permute again and again (let's say thousands of times) instead of having a Signature we have a Signature Matrix

Element	S1	S2	S3	S4
a	1	0	0	1
b	0	0	1	0
d	1	0	1	1
e	0	0	1	0
c	0	1	0	1

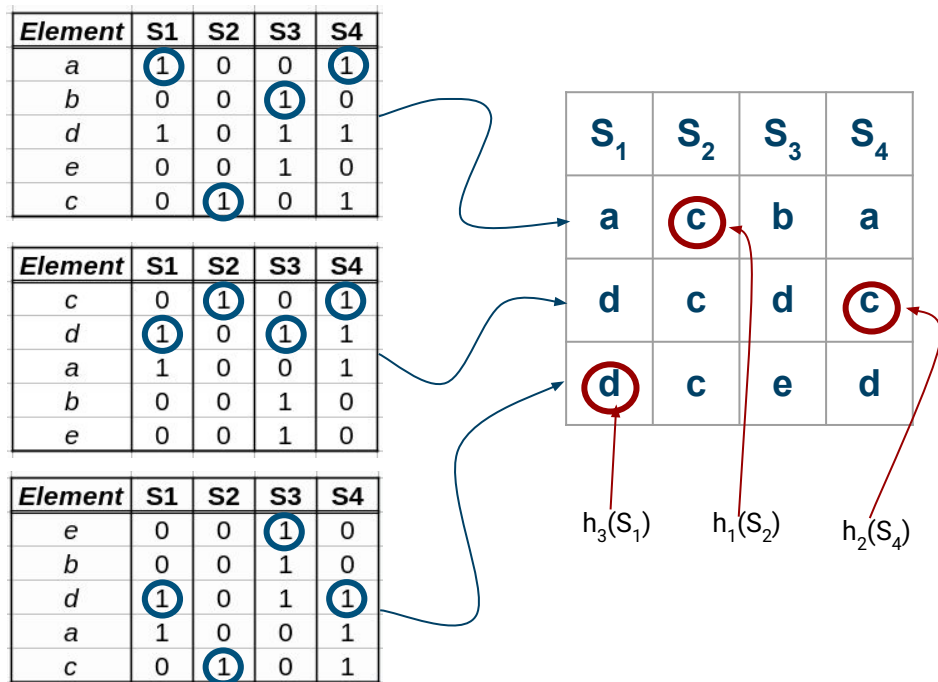
Element	S1	S2	S3	S4
c	0	1	0	1
d	1	0	1	1
a	1	0	0	1
b	0	0	1	0
e	0	0	1	0

Element	S1	S2	S3	S4
e	0	0	1	0
b	0	0	1	0
d	1	0	1	1
a	1	0	0	1
c	0	1	0	1

S ₁	S ₂	S ₃	S ₄
a	c	b	a
d	c	d	c
d	c	e	d

Signature Matrix

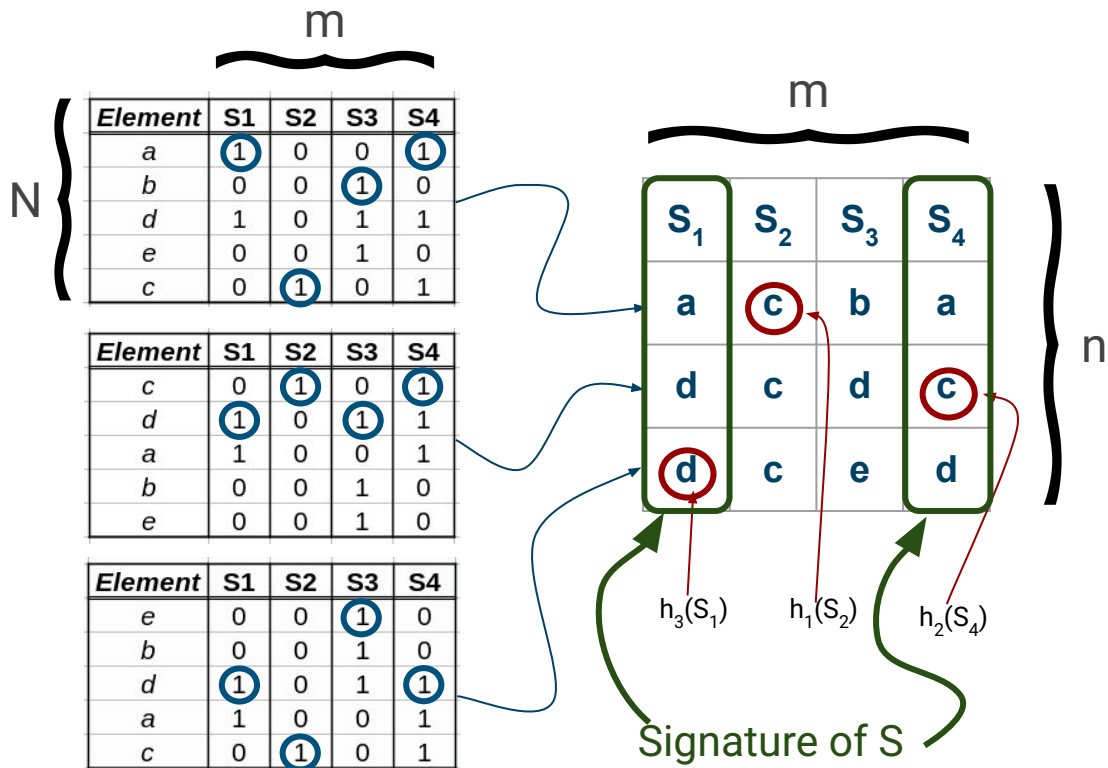
If we consider $h_1, h_2, \dots, h_n$ the permutations, the Signature Matrix is made by m columns (the same number of columns in the Characteristic Matrix) where each element of the column is the minhash value for a permutation.



Signature Matrix

Note that the signature matrix has the same number of columns as Characteristic Matrix M but only n rows. Even if M is not represented explicitly it is normal for the signature matrix to be much smaller than M .

$$N \gg n$$



Characteristic Matrix vs Signature Matrix

- Characteristic Matrix is much larger than Signature Matrix (same column but more rows)
- By the theorem, the Jaccard Similarity between the Shingle sets of two documents is equal to the probability that two columns have the same value in a given row of the signature matrix
- Since the permutations were chosen independently, we can think of each row of the signature matrix as an independent experiment
- The expected number of rows in which two columns agree equals the Jaccard similarity of their corresponding sets

The error of the Signature

The more the number of rows in the signature matrix, the smaller the expected error in the estimate of the Jaccard similarity will be

Computing Minhash Signature

When the Characteristic Matrix is wide (million or billion of rows), even picking a random permutation can be very time-consuming.

Luckily it is possible to simulate the effect of a random permutation by a random hash function h . So we can choose n different hash functions $(h_1, h_2, \dots, h_n)$ and each of them will represent a permutation.

An example of hash function $h(x)$ can be

$$h(x) = 3x + 1 \bmod 5.$$

Then, for the row having index 4, the $h(4) = 3$

Computing Minhash Signature

Let see the algorithm for building the Minhash Signature Matrix

On the left side, there is the Characteristic Matrix (we have just replaced the label for the rows with an integer).

On the right the assigned value depending on two hash functions

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

Computing Minhash Signature

We start initializing the signature matrix with infinite values

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

Computing Minhash Signature

Then we start for row 0 in the Characteristic Matrix. We see 1 in column S_1 and S_4 . As $h_1(S_1)$, $h_2(S_1)$, $h_1(S_4)$, $h_2(S_4)$ are less than infinite, we replace infinite with the corresponding value h_1 and h_2 on the Signature Matrix only on all rows in the column S_1 and S_4

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

Computing Minhash Signature

Then we move on row 1 in the Characteristic Matrix. We see that only S_3 column has 1, so we replace $h_1(S_3)$ and $h_2(S_3)$ in the Signature with respectively 2 and 4, being such values less than infinite.

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

Computing Minhash Signature

Then we move on row 2 in the Characteristic Matrix. We see that S_2 and S_4 columns have 1, so we replace $h_1(S_2)$ and $h_2(S_2)$ in the Signature with 3 and 2, being such values less than infinite, but we do not replace $h_1(S_4)$ and $h_2(S_4)$ because the respective values (3 and 2) are greater than the current value in the Signature (1 and 1)

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

Computing Minhash Signature

Next row 3. In the Characteristic Matrix only columns S_1 , S_3 and S_4 have 1, so in the Signature we can put the corresponding value of h_1 and h_2 (that is 4 and 0) on all three columns only if the such values are lower than the value already stored in the Signature Value

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	0	2	0	0

Computing Minhash Signature

Next row number 4. In the Characteristic Matrix only columns S_3 has 1, so in the Signature we can put the corresponding value of h_1 and h_2 (0 and 3) on all three columns only if the such values are lower than the value already stored in the Signature Value

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	0	2	0	0

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Jaccard Similarity on Signature Matrix

This is the final Minhash Signature Matrix.

Underneath instead is the original
Characterization Matrix

By using the Signature Matrix, we can compute similarities between two documents. For example S1 and S4 seem to be very similar (they have $J(S1, S4) = 1$). In the original matrix instead

2/3

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Element	S1	S2	S3	S4
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0

Jaccard Similarity on Signature Matrix

The difference in the Jaccard Similarity (computed on the Signature Matrix) and the **true** Jaccard Similarity (computed on the Characteristic Matrix) is because the fraction of rows in the Signature Matrix is only **an estimate of the true Jaccard similarity**, and this example is much too small **for the law of large numbers** to assure that the estimates are close

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Element	S1	S2	S3	S4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Dictionary Compression

Dictionary Compression

If compared with the Posting Lists, Dictionary is very small. So, why is it important to compress it if it is responsible for only a **small percentage of the overall space** requirements of the IR system?

Because a smaller dictionary can stay into the main memory, and this means **less disk seeks** required to process a query.

To allow the dictionary **to stay into the memory**, we need to compress it.

Statistic-driven Compression

Collection for Reuters-RCV1	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

Dictionary as a string

The simplest way to manage a Dictionary is by storing terms into the vocabulary and **sort them lexicographically**.

Each term is stored in **fixed size of 20 bytes** (only few english words require more than 20 chars).

We also need 4 bytes for the **document frequency** and 4 byte to **point to the Posting Lists**

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→
20 bytes	4 bytes	4 bytes

$$D_{size} = T(20 + 4 + 4) = 28T$$

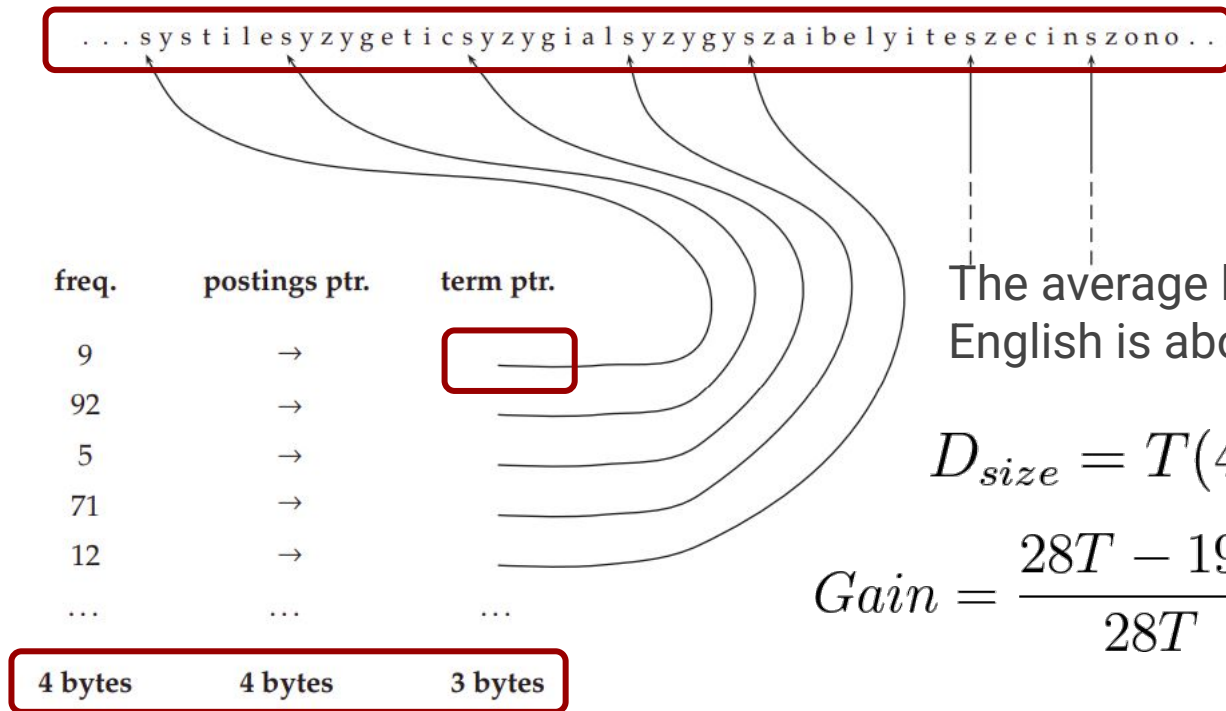
Dictionary as a string

But:

- The average length of a term in English is about **8 characters**, having a fixed size length of 20 chars we lose **12 chars**
- $D_1 = 28T$ $D_2 = 16T$ **Gain** = $(D_1 - D_2)/D_1 = (28T - 16T)/28T \approx 42.86\%$
- We cannot manage longer words

Solution: storing the dictionary terms as one long string of characters

Dictionary as a string



The average length of a term in English is about **eight characters**

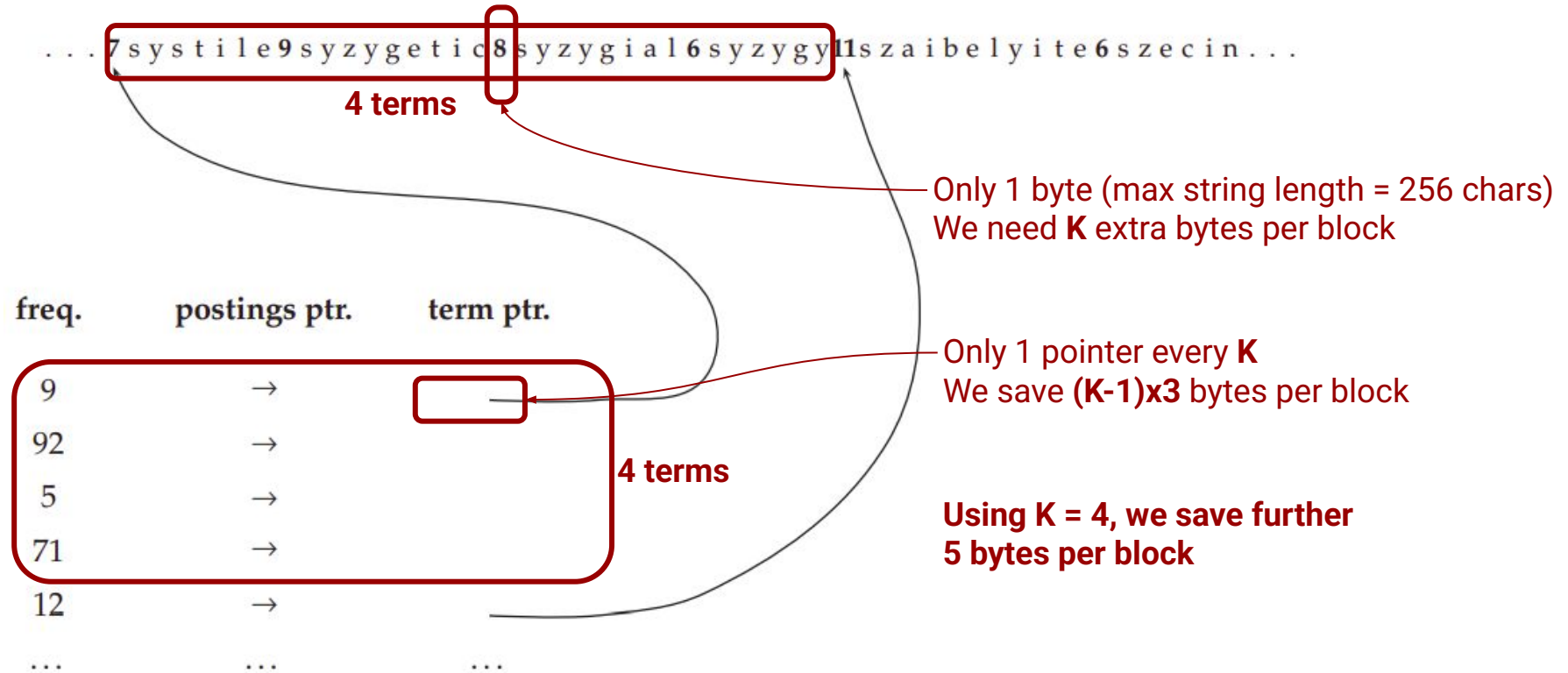
$$D_{size} = T(4 + 4 + 3 + 8) = 19T$$

$$Gain = \frac{28T - 19T}{28T} = \frac{9T}{28T} \approx 0.32 = 32\%$$

Dictionary as blocks of strings

We can even compress the size of the dictionary by **reducing the number of pointers** and managing the long string as **blocks of strings**

Dictionary as blocks of strings



Dictionary as block of strings

The diagram illustrates the formula for dictionary size, D_{size} , with annotations for each part of the expression:

- $\frac{T}{K}$: number of blocks
- K : document frequency
- 4 : postings pointer
- 4 : term average length
- 8 : pointer to block string
- 1 : auxiliary byte for term length
- 3 : pointer to block string

$$D_{size} = \frac{T}{K} (K(4 + 4 + 8 + 1) + 3) = \frac{T}{K} (17K + 3)$$

Dictionary as block of strings

If $K = 4$, then we can get a gain equals to

$$\begin{aligned} Gain &= \frac{28T - \frac{T}{K} (17K + 3)}{28T} = \frac{28T - \frac{T}{4} (17 * 4 + 3)}{28T} \\ &= \frac{28T - 17.75T}{28T} \approx 36.61\% \end{aligned}$$

Tradeoffs

- Managing Dictionary as String or block of string is **more complex**
- The larger is K , the more is the compression factor
- But it takes more time in search
 - because we need to scan the whole block

Dictionary with Front Coding

Observation: One block in blocked compression ($k = 4$) ...
8automata8automate9automatic10automation

As the example shows, when terms are sorted alphabetically, they more often have the **same prefix**.

We can exploit this observation in order to further compress the Dictionary

Dictionary with Front Coding

How does it work?

We can use special character to identify the common prefix

One block in blocked compression ($k = 4$) ...
8automata8automate9automatic10automation



...further compressed with front coding.
8automat*a1♦e2♦ic3♦ion

Comparing all techniques

Collection for Reuters-RCV1	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9

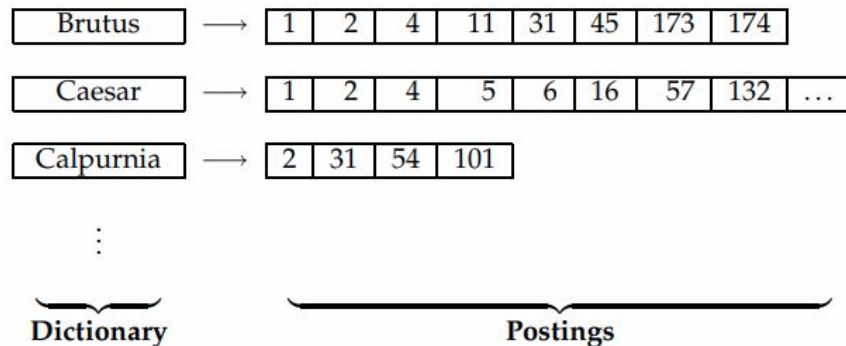
$$Gain = \frac{11.2MB - 5.9MB}{11.2MB} \approx 0.47 = 47\%$$

Postings List Compression

How to reduce posting list size?

Let consider the simplest Posting Lists:

- Each posting is the DocID
- Each ID can be identified by 32 bits
- Using 32 bit we can identify 2^{32} documents (sometimes too much)

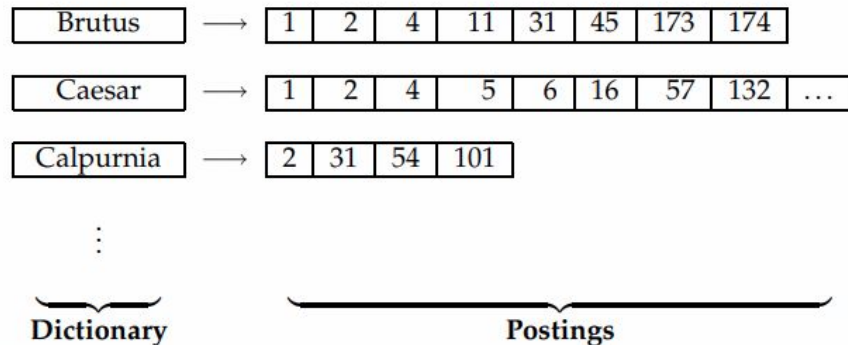


We can use $\log_2 D$ bits for the DocID

How to reduce posting list size?

But we can do better.

Observation: terms that are very frequent appear often in the documents. This means the DocIDs, being sorted, are very **close to each other**.



How to reduce posting list size?

So, instead of storing the DocIDs, why don't we store only the **gap**?

For very recurring Terms, the gap is very small and require a small number of bits

So, instead of $\log_2 D$ bits for the DocIDs, we can use fewer bit but **we cannot know in advance how large is this gap** and then how many bits are required to store the gap.

For this reason, we need a **Variable Byte Encoding**.

Variable Byte Code

A Variable Byte Code is a sequence of 8 bit (that is a byte), where:

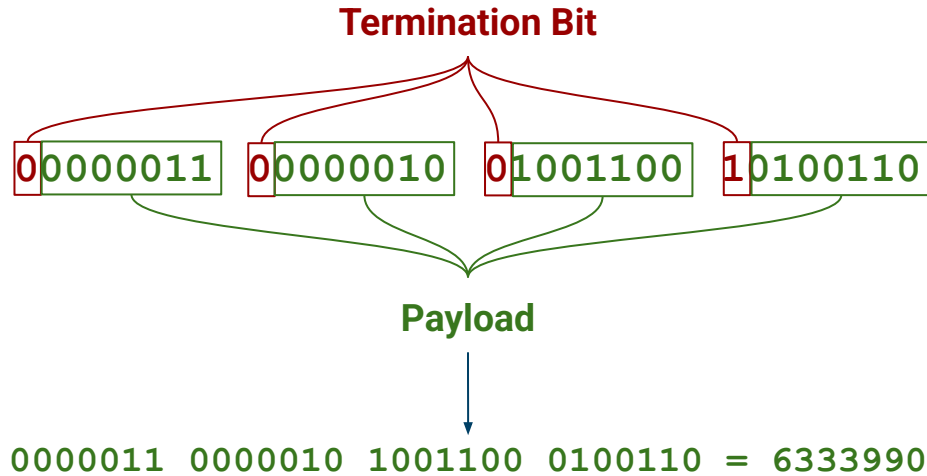
- the first bit is named **termination bit**
- the last seven bits are named **payload**

One or more Variable Byte Codes can be used to describe the gap

The termination Bit

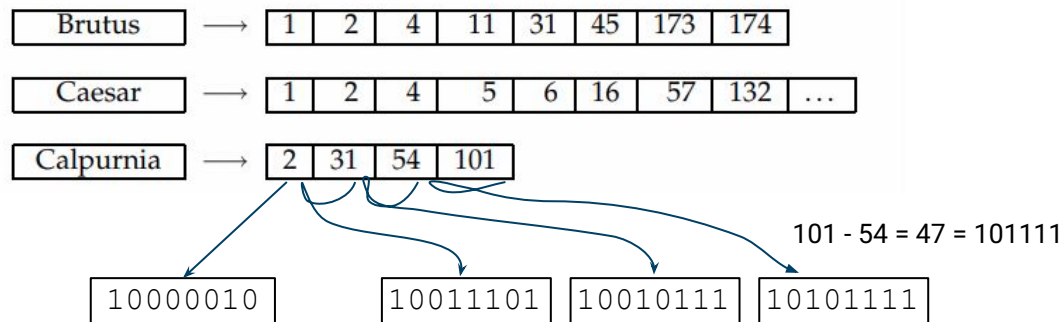
- is set to 0 if the next byte is the continuation of the same Variable Byte Code
- is set to 1 if this byte is the last byte of the Variable Byte Code

Variable Byte Code



Variable Byte Code

For example: suppose we have 65,000 documents. We need 16 bits for each DocID. Let's consider the last Postings List using Variable Byte Code:



Variable Byte Code

Consideration: if the gap was 127, we can represent the gap using only one byte:

11111111

but if the gap was 128 (only one higher), we need to use two bytes:

00000001 **1**0000000

The first 6 zeros are useless

Tradeoffs of Variable Byte Code

Variable Byte Code represents a **bytewise compression**. This means that the gaps are always represented by one or more bytes

Working with a **bitwise compression method**, we can achieve better compression ratio: **Gamma Codes**

Gamma Code

Gamma Code is a bitwise compression method.

It represents the gap with a sequence of bits having an **adaptive length**

The Gamma Code is made by two parts:

- the offset: is the binary representation of the gap **without the heading bit**
- the length: is an **unary code** of the length
 - the unary code of n is a string of n ones followed by a zero, example $5 = 111110$

Gamma Code

An example:

Suppose we want to represent 13 (decimal) in Gamma Code:

- $13_{10} = 1101_2$
- offset = **101**
- $\text{length}(\text{101}) = 3 = \text{1110}$
- $\text{Gamma}(13) = \text{1110101}$

Gamma Code

What is the maximum length of a Gamma Code?

If G is the gap:

- the length of the **offset** $\lfloor \log_2 G \rfloor$
- the length of the **length** $\lfloor \log_2 G \rfloor + 1$

So the maximum length $2 \lfloor \log_2 G \rfloor + 1$

Comparing all techniques

Collection for Reuters-RCV1	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

Data Structure	Size in MB
Term Incidence Matrix	40,000
Postings with no compression using 32-bit for DocID	400
Postings with no compression 20-bit for DocID	250
Postings with variable byte encoding	116
Posting with gamma encoding	101

Comparing all techniques

Data Structure	Size in MB	Gain
Term Incidence Matrix	40,000	
Postings with no compression using 32-bit for DocID	400	
Postings with no compression 20-bit for DocID	250	$(400 - 250) / 400 = 0.375 =$ 37.50%
Postings with variable byte encoding	116	$(400 - 116) / 400 = 0.71 =$ 71.00%
Posting with gamma encoding	101	$(400 - 101) / 400 = 0.7475 =$ 74.75%